Estructura de Datos

Tema 3:

Pilas

GRADO EN INGENIERÍA DEL SOFTWARE

Salvador Sánchez y Cristian Rodríguez

Universidad
Rey Juan Carlos

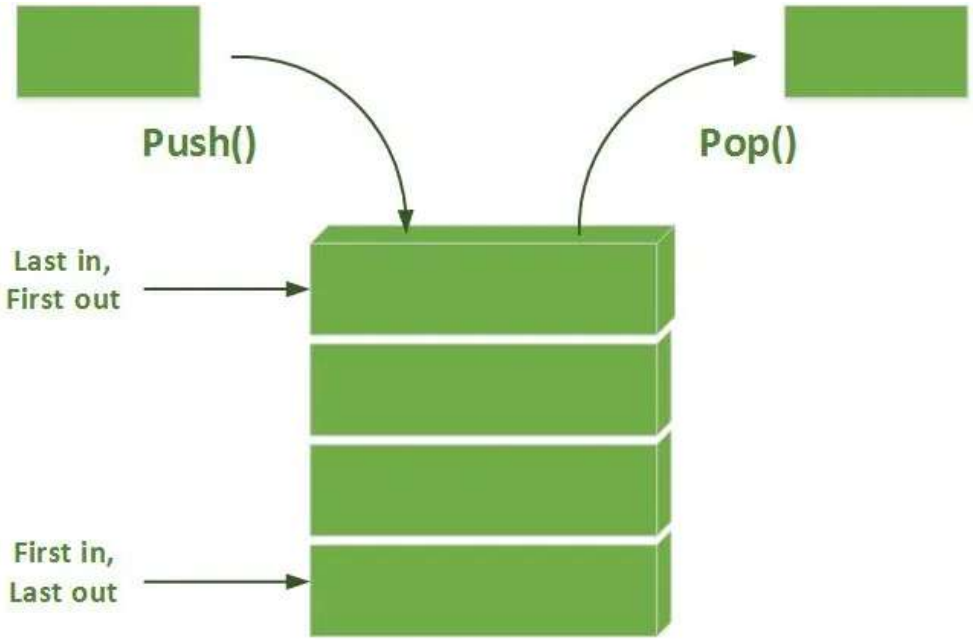
Curso 2023-2024

Contenidos

- Definición
- El TAD Pila: operaciones básicas
- Implementación
- Ejemplos de aplicación
- Pilas y genericidad
- Ejercicios propuestos

Pila: definición

- Estructura de datos lineal, dinámica, que da soporte al concepto **LIFO**: **L**ast **I**n, **F**irst **O**ut

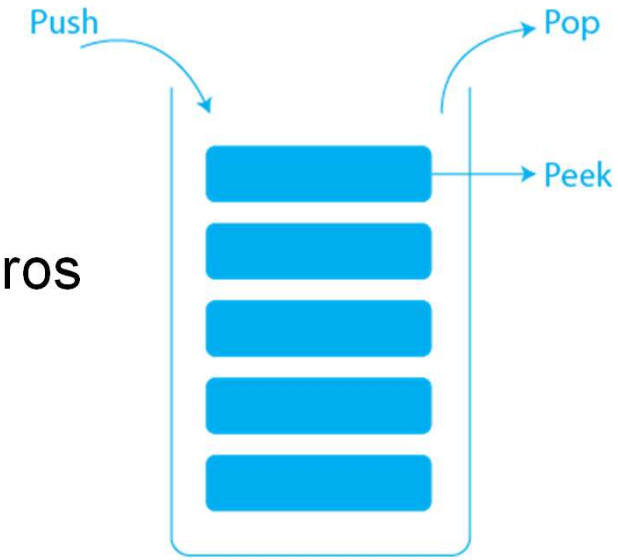


TAD Pila

- Independientemente de su implementación, una pila debe proporcionar al menos 4 operaciones básicas:
  - Meter un elemento (`push`)
  - Sacar un elemento (`pop`)
  - Consultar si está vacía (`is_empty`)
  - Conocer el elemento de la cima (`peek`/`top`)
  - \+ Inicializar/crear

TAD Pila: especificación

- **Nombre:** Pila.
- **Valores posibles:** *vacía, otros (según elementos)*.
- **Operaciones:**
  - `push`
  - `pop`
  - `is_empty`
  - `peek`
- **Uso de otros tipos:** tipo base, arrays / punteros (según implementación)

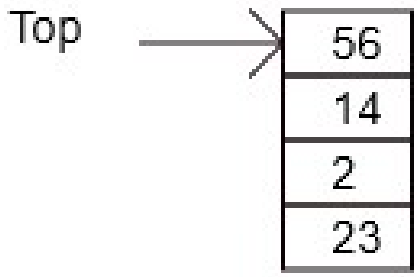


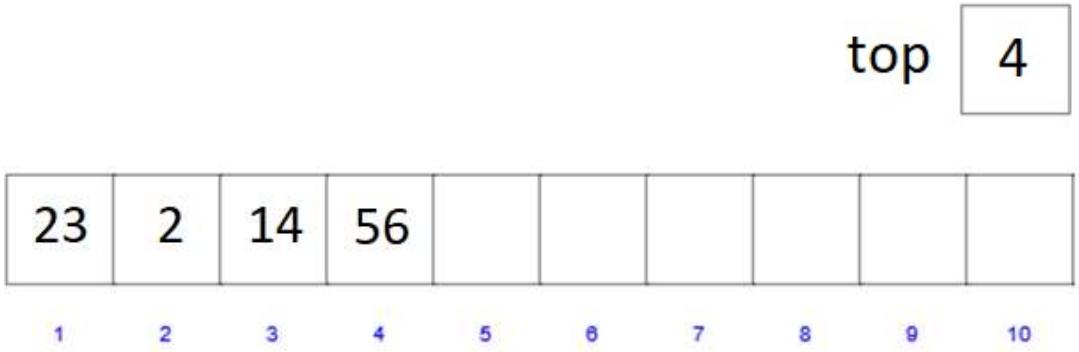
Implementación

- Dos formas habituales de implementarla: con un **array** o utilizando **memoria dinámica**
  - Externamente no debería ser posible distinguirlo
- Utilizando arrays hay limitaciones de espacio, pero la implementación es muy sencilla
- Utilizando punteros y memoria dinámica no hay límite de espacio (en teoría) pero la implementación es algo más compleja

Implementación con un array

- Necesitaremos una variable entera que indicará la posición de la cima + un array del tipo base a almacenar





Implementación con un array

- Utilizaremos un array cuyo índice empiece por 1
- Habrá un máximo de elementos que estableceremos de manera estática (constante MAX_ELEMENTOS)
- Necesitaremos una variable entera para registrar la posición del elemento cima (0 si está vacía)
- Antes de insertar o borrar un elemento hay que comprobar si ello es posible (precondiciones)
- Al sacar un elemento, no haremos un borrado físico del mismo en el array, simplemente decrementaremos el índice del elemento cima.
- La inserción y borrado tienen una complejidad O(1): muy eficiente.

Implementación con un array

```
const MAX_ELEMENTOS = 100;
type
  tPilaEnteros = record
    cima: integer;
    stack: array[1..MAX_ELEMENTOS] of integer;
  end;
procedure initialize(var p:tPilaEnteros);
procedure push(var p:tPilaEnteros; x:integer);
function pop(var p:tPilaEnteros): integer;
function peek(p:tPilaEnteros): integer;
function isEmpty(p:tPilaEnteros): boolean;
function isFull(p:tPilaEnteros): boolean;
procedure clear(var p:tPilaEnteros);
```

Contrato

Implementación con un array

```
procedure initialize(var p:tPilaEnteros);
begin
  p.cima := 0;
end;

function isEmpty(p:tPilaEnteros): boolean;
begin
  isEmpty := p.cima = 0;
end;

function peek(p:tPilaEnteros): integer;
begin
  if not isEmpty(p) then
       peek := p.stack[p.cima];
end;
```

Implementación con un array

```
procedure push(var p:tPilaEnteros; x:integer);
begin
  if p.cima < MAX_ELEMENTOS then
  begin
    p.cima:= p.cima + 1;
    p.stack[p.cima]:= x;
  end
end;

function pop(var p:tPilaEnteros): integer;
begin
  if not isEmpty(p) then
  begin
    p.cima := p.cima - 1;
    pop := p.stack[p.cima+1];
  end;
```

Funcionalidades adicionales

- Podemos ampliar nuestro TAD Pila con algunas operaciones extra tales como:
  - `is_full()` → Comprueba si está llena
  - `clear()` → Vacía la pila

- Se proponen como ejercicio.

Funcionalidades adicionales

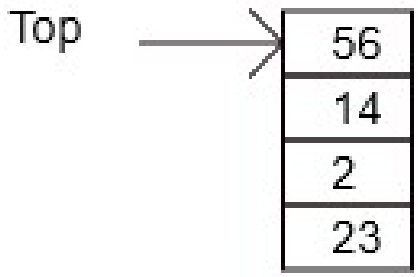
```
{*** Funciones adicionales *** }

function isFull(p:tPilaEnteros): boolean;
begin
  isFull := p.cima = MAX;
end;

procedure clear(var p:tPilaEnteros);
begin
  p.cima := 0;
end;
```

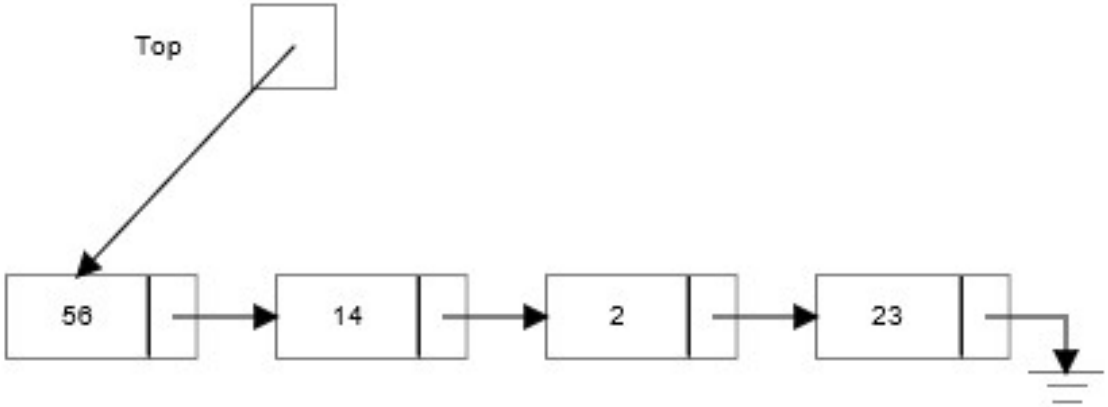
Pila implementada con array (resumen)

- **Desventaja**: es necesario conocer el tamaño a priori.
- Es necesario establecer **precondiciones** en varias operaciones cuya implementación más lógica sería utilizar excepciones:
  - Pascal no da soporte eficiente a este concepto
  - Las implementaciones propuestas dejan abierta la puerta a comportamientos imprevistos si no se emplean correctamente
  - Es posible una reflexión diferente para cada tipo base

Implementación en memoria dinámica

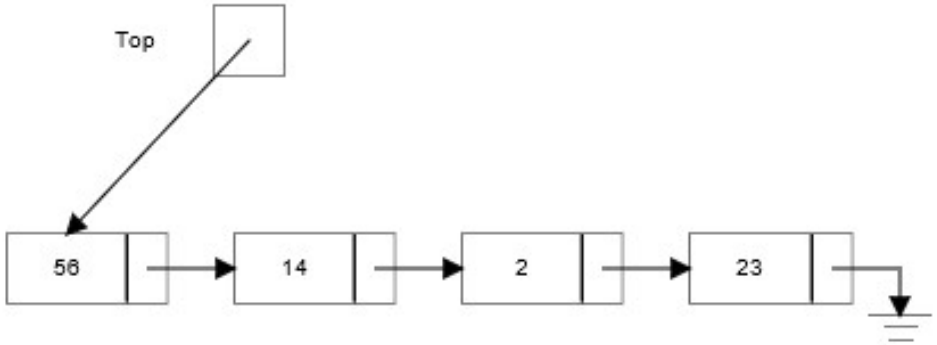
- Necesitaremos un tipo “nodo” para representar cada elemento apilado y un puntero a la cima.





Implementación en memoria dinámica

- Utilizaremos una estructura auxiliar “nodo” con dos partes: la información (del tipo base) y un puntero al nodo anterior (el que está debajo de él en la pila o `nil` si no hay ninguno más)
- Al insertar un elemento hay que crear un nuevo nodo y poner la pila a apuntar al mismo, ya que entra arriba del todo
- No hay que comprobar overflow pues no tiene límite de almacenamiento, solo underflow (`is_empty`)



Implementación en memoria dinámica

```
type
  tnodo = record
    info: integer;
    ant: ^tnodo;
  end;
  tPilaEnteros = ^tnodo;
```

Contrato

```
procedure initialize(var p:tPilaEnteros);
procedure push(var p:tPilaEnteros; x:integer);
function pop(var p:tPilaEnteros): integer;
function peek(p:tPilaEnteros): integer;
function isEmpty(p:tPilaEnteros): boolean;
procedure clear(var p:tPilaEnteros);
```

Implementación en memoria dinámica

```
procedure initialize(var p:tPilaEnteros);
begin
  p := nil;
end;

function isEmpty(p:tPilaEnteros): boolean;
begin
  isEmpty := p = nil;
end;

procedure push(var p:tPilaEnteros; x:integer);
var
  new_node: ^tnodo;
begin
  new(new_node);
  new_node^.info := x;
  new_node^.ant := p;
  p:= new_node;
end;
```

Implementación en memoria dinámica

```
function pop(var p:tPilaEnteros): integer;
var
  elem: integer;
  aux: ^tnodo;
begin
  if not isEmpty(p) then
  begin
    elem := p^.info;
    aux := p;
    p := p^.ant;
    dispose(aux);
  end;
  pop := elem;
end;

function peek(p:tPilaEnteros): integer;
begin
  if not isEmpty(p) then
       peek := p^.info;
end;
```

Funcionalidades adicionales

- Podemos ampliar nuestro TAD Pila con alguna operación extra tal como:
  - `clear()` → Vacía la pila
- Se propone como ejercicio.

Implementación en memoria dinámica

```
{*** Funciones adicionales *** }

procedure clear(var p:tPilaEnteros);
begin
  while not isEmpty(p) do
    pop(p);
end;
```

Aplicaciones

- Muy utilizadas en computación para un gran número de aplicaciones tales como:
  - Evaluación de expresiones
  - Llamadas a subprogramas
  - Guardar listado de órdenes para deshacer-rehacer en editores
  - Adelante-atrás en navegadores web
  - Algoritmos de Backtracking (ej. Colocar 9 reinas en un tablero de ajedrez, búsqueda de salida en un laberinto, etc.)
  - Gestión de memoria
  - Etc.

Pilas y genericidad

Limitaciones del enfoque anterior

- Hasta ahora hemos visto una aplicación específica de las pilas: pila de enteros
- Las pilas de otros elementos tienen idéntico comportamiento, solo cambia el tipo base
- **Genericidad**: Permite programar las operaciones de pila utilizando como elemento base un tipo genérico no especificado y luego especificar en cada uso.
  - A las estructuras genéricas se las conoce esto como “plantillas” (*templates*)
  - Pascal no incluye soporte para plantillas

(pseudo) genericidad en Pascal

Hay dos alternativas:

- Escribir una estructura similar para cada tipo de dato que utilicemos: pila_real, pila_registros…
  - No parece una práctica muy productiva
  - Si hay registros, tener en cuenta si tiene varios campos pues no se puede p.ej asignar completo
- Programar una pila genérica.
  - Programar una unidad Tpila que requiera otra unidad para el tipo base genérico “TipoElemento”.
  - No es la solución óptima, pero sí algo más parecido a lo ideal

TElemento

- Cada vez que necesitemos pilas de distintos tipos, habrá que definir una nueva unidad `TElemento`
- Dentro de la unidad `TElemento`, creamos un nuevo tipo definido que será el tipo base de nuestra pila.
  - Ej. si necesitamos una pila de personas, `TElemento` exportará el tipo `Persona` y sus operaciones.
- Las operaciones de asignación y comparación deben definirse en la unidad `TElemento`:
  - `assign(var e: TElemento; e2: TElemento);`
  - `lessThan(e: TElemento; e2: TElemento): boolean;`
  - `equalTo(e: TElemento; e2: TElemento): boolean;`
  - …

TElement: Ejemplo

```
UNIT TElement;
TYPE
    TElement = record
        data1: integer;
        data2: string[50];
    end;
    procedure assign( var e1: TElement; e2: TElement );
    begin
        e1.data1 := e2.data1;
        e1.data2 := e2.data2;
        ...
    end;
```

Pila genérica (con arrays)

```
UNIT UStack;
USES TElement;
INTERFACE
    CONST MAX_ELEMS = 100;

    TYPE
        TStack = RECORD
            top: TElement;
            stack: ARRAY[1..MAX_ELEMS] of TElement;
        end;

    procedure initialize( VAR p: TStack );
    procedure push( VAR p: TStack; x: TElement );
    function pop( VAR p: TStack ): TElement;
    function peek( p: TStack ): TElement;
    function isEmpty( p: TStack ): boolean;
```

Pila genérica (con arrays)

```
function peek( p: TStack ): TElement;
begin
    if not isEmpty( p ) then
        assign( peek, p.stack[ p.top ] );
        peek := p.stack[ p.top ];
end

procedure push( VAR p: TStack; x: TElement );
begin
    if( p.top < MAX_ELEMS ) then
    begin
        p.top := p.top + 1;
        assign( p.stack[ p.top ], x );
        p.stack[ p.top ] := x;
    end;
end;
```

Pila genérica (con memoria dinámica)

```
UNIT UStack;
USES TElement;
INTERFACE
    TYPE
        TNode = RECORD
            info: TElement;
            ant: ^TNode;
        end;
        TStack = ^TNode;

    procedure initialize( VAR p: TStack );
    procedure push( VAR p: TStack; x: TElement );
    function pop( VAR p: TStack ): TElement;
    function peek( p: TStack ): TElement;
    function isEmpty( p: TStack ): boolean;
```

Contrato

Pila genérica (con memoria dinámica)

```
procedure initialize( var p: TStack );
begin
    p := NIL;
end;

function isEmpty( p: TStack ): boolean;
begin
    isEmpty := p = NIL;
end;

procedure push( VAR p: TStack; x: TElement );
VAR
    new_node: ^TNode;
begin
    new( new_node );
    assign( new_node^.info, x );
    new_node^.ant := p;
    p := new_node;
end;
```

Pila genérica (con memoria dinámica)

```
function pop( VAR p: TStack ): TElement;
VAR
    aux: ^TNode;
begin
    if not isEmpty( p ) then
    begin
        assign( pop, p^.info )
        aux:= p;
        p := p^.ant;
        dispose( aux );
    end;
end;

function peek( p: TStack ): TElement;
begin
    if not isEmpty( p ) then
        assign( peek, p^.info );
end;
```

Funcionalidades adicionales

- Como antes, podemos ampliar nuestro TAD Pila genérico con operaciones extra tales como:
  - `is_full()` → Comprueba si está llena
  - `clear()` → Vacía la pila

- De nuevo se proponen como ejercicio.

Ejercicios propuestos

Ejercicio propuesto 1

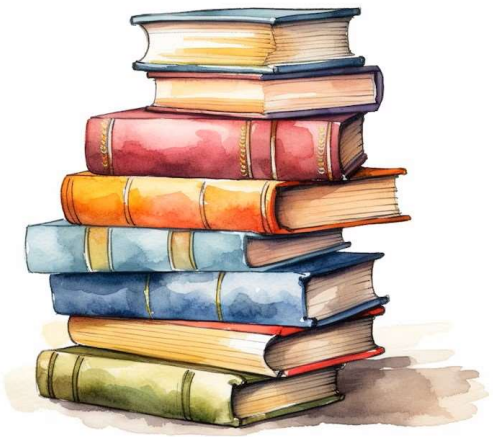
- Utilizando una pila, comprobar si una expresión que contiene paréntesis está balanceada o no.
- Ejemplos:
  - 3 + (8 * 9 + (34 – 2) – 72/ (3 – 56)) → OK
  - 3 + 8 * 9 + (34 – 2) – 72/ (3 – 56)) → NO OK

Ejercicio propuesto 1-bis

- Utilizando una pila, comprobar si una expresión que contiene paréntesis y corchetes está balanceada o no.
- Ejemplos:
  - 3 + [(8 * 9 + (34 – 2) – 72/ (3 – 56))] → OK
  - [3 + 8 * 9 + (34] – 2) – 72/ (3 – 56)) → NO OK

Ejercicio propuesto 2

- Implementar una pila de libros, guardando de cada libro su título, autor, ISBN, idioma y número de páginas.
- Una vez implementado lo anterior, implementa un subprograma que recoloque los libros de la pila al revés de como están actualmente.



Ejercicio propuesto 3

- Ampliar la especificación del TAD Pila de enteros añadiendo las siguientes operaciones e implementarlas en ambos enfoques de implementación (con array y con memoria dinámica):
  - `count(pila)`: retorna cuántos elementos tiene la pila
  - `bottom(pila)`: consulta el elemento más profundo de la pila
  - `mount(pila1,pila2)`: poner todos los elementos de la segunda pila encima de la cima actual de la primera respetando el orden de los elementos
  - `npop(pila,n)`: quita tantos elementos de la pila como indica el parámetro entero
  - `summarize(pila,m)`: reemplaza los primeros m elementos de la pila por su suma. Si hay menos de m elementos reemplaza todos los que haya por su suma.

Ejercicio propuesto 4

- Utilizando una pila, comprobar si una frase es un palíndromo (se lee igual al derecho y al revés).
- Ejemplos: